Opseeq: A Unified Architecture for Local Superintelligent Desktop Agency

Integrating Triadic Cognition, Bounded Reasoning, Artifact Memory, Observability, and Post-Quantum Attestation into a Single Locally Executable Intelligence Framework

Dylan Kawalec
chaotic beauty LLC

March 2026 — Master Framework v1.0

Abstract

This paper presents the official unified architecture of **Opseeq**, a local-first desktop intelligence framework that transforms a developer's workstation into a sovereign, formally grounded, and operationally coherent reasoning system. Opseeq is not a single model, a chatbot wrapper, or a cloud-dependent copilot. It is a layered computational architecture comprising a *triadic cognitive core* (Context Engine, Process Engine, Query Engine), a *bounded hierarchical reasoning controller* (HPC-GoT), an *artifact-centric memory runtime* (Cellar), a *sidecar observability and optimization layer* (TraceRank), and a *post-quantum attestation substrate* (RQLH and Quantum-Weave). The architecture is orchestrated through a disciplined OODA pipeline, operates primarily on local open-weight models, and produces auditable artifacts rather than opaque outputs.

This document synthesizes ten prior technical papers into a single internally consistent framework. Every major subsystem is defined, positioned within the architecture, and connected to the others through explicit interfaces, shared state models, and formal dependencies. Where the source papers contained contradictions—in symbol usage, deployment assumptions, threshold regimes, or determinism claims—this paper resolves them. The result is the canonical reference for Opseeq as an integrated intelligence architecture for desktop agency, orchestration, memory, reasoning, and system execution.

Contents

1 Introduction **3**
1.1 Scope and Claim Discipline . . . 3

2 Problem Framing and Motivation **3**

3 Unified Theoretical Foundations **4**
3.1 Canonical Notation . . . 4
3.2 Core Axioms . . . 5
3.3 Core Postulates . . . 5
3.4 Key Definitions . . . 6
3.5 Lemmas and Corollaries . . . 6

4 Full Architectural Overview **7**

5 Layer I: Inference and Runtime **7**

6 Layer II: The Triadic Cognitive Core **7**
6.1 Context Engine . . . 8
6.2 Process Engine . . . 8
6.3 Query Engine . . . 9
6.4 Triadic Integration . . . 9

7 Layer III: Reasoning and Search **9**
7.1 The OODA Orchestration Pipeline . . . 9
7.2 HPC-GoT: The Bounded Reasoning Controller . . . 10
7.3 Reconciliation: Determinism vs. Stochastic Inference . . . 10

8 Layer IV: Memory and Artifacts **10**
8.1 The CELLAR Runtime . . . 10
8.2 Hot and Cold Storage Planes . . . 11
8.3 Deduplication Pipeline . . . 11

9 Layer V: Observability and Optimization **11**
9.1 TraceRank . . . 11
9.2 Adapter Promotion Pipeline . . . 12

10 Layer VI: Security and Attestation **12**
10.1 RQLH Consensus . . . 12
10.2 Quantum-Weave: Post-Quantum Data Security . . . 12
10.3 Trust Boundary Model . . . 12

11 Layer VII: Integration — OpenClaw, Mermate, and Synth **12**
11.1 OpenClaw . . . 12
11.2 Mermate . . . 13
11.3 Synth (Synthesis-Trade) . . . 13
11.4 Gateway Implementation . . . 13

12 Assembly and Implementation Logic **13**
12.1 How the System Is Constructed . . . 13
12.2 Runtime Wiring . . . 14
12.3 The Assembly Loop . . . 14

13 Temporal State Sequencing and System Behavior Over Time **14**

14 Human-in-the-Loop and Tool-Mediated Control **15**
14.1 The Human Invariant Layer . . . 15
14.2 Guardrailed Tool Execution . . . 15
14.3 Tiered Validation Policy . . . 15

15 Performance and Local Runtime Model **15**

16 Reliability and Production Constraints **16**

17 Final Synthesis **16**

A Source Cross-Reference Matrix **17**

B Unresolved Source **17**

C Notation Index **18**

1 Introduction

The prevailing model for AI-assisted work treats the language model as the entire system: a request enters, a response exits, and the path between them is opaque. This paradigm scales poorly to serious engineering, where correctness must be traceable, reasoning must be auditable, memory must persist across sessions, and execution must be safe.

Opseeq proposes a different architecture. The language model is not the system; it is the *policy layer* within a larger computational runtime. Around it, Opseeq constructs a disciplined cognitive stack: engines for context, process, and query management; a bounded reasoning controller for structured exploration; a durable artifact memory with hot and cold storage planes; a sidecar observability layer that detects structural degradation before quality visibly declines; and a cryptographic attestation substrate that ensures integrity under post-quantum threat models.

The framework is local-first. It is designed to run on a developer's hardware using open-weight models served through Ollama, vLLM, or llama.cpp, with optional cloud inference treated as an out-of-trust supplement. It is artifact-centric: every expensive operation produces a durable, typed, provenance-preserving record. It is formally grounded: its subsystems are defined through axioms, postulates, lemmas, and explicit operator algebras rather than informal descriptions.

This paper is the result of unifying ten prior technical documents, each of which described a partial view of the Opseeq architecture. The unification is not mechanical. It required resolving symbol collisions, reconciling contradictory assumptions about determinism and deployment topology, harmonizing multiple threshold regimes into a single tiered validation policy, and constructing explicit interface contracts between subsystems that were previously described in isolation.

1.1 Scope and Claim Discipline

Following the claim-ladder methodology introduced in the TraceRank paper[1], every assertion in this document is implicitly one of:

Normative
: The official Opseeq architectural design.

Implemented
: Present in the current codebase and operationally verified.

Specified Formally defined but not yet implemented.

Roadmap Planned but not yet formally specified.

Where the distinction matters, it is made explicit.

2 Problem Framing and Motivation

Desktop AI systems today suffer from five structural deficiencies that Opseeq addresses:

1. **Statelessness.** Most assistants treat each interaction as independent, discarding the accumulated context of prior work.

2. **Unverifiable reasoning.** The path from input to output is internal to the model and cannot be externally audited or validated.

[1] Source: `tracerank.tex`, §Claim ladder.

3. **Unsafe execution.** When systems interact with filesystems, terminals, and APIs, they do so with weak or opaque control surfaces.

4. **Cloud dependency.** High-capability systems typically require remote inference, introducing privacy, latency, and availability constraints.

5. **Ephemeral computation.** Expensive reasoning vanishes after generation. There is no durable record of hypotheses considered, evidence gathered, or branches pruned.

Opseeq's architectural response is to make *structured process* a first-class system property rather than an emergent behavior of prompting. The framework transforms workflow optimization into a general intelligence architecture for desktop execution, coordination, and reasoning.

3 Unified Theoretical Foundations

The theoretical structure of Opseeq is built from the formal constructs distributed across the source corpus. This section unifies them into a single consistent foundation.

3.1 Canonical Notation

To resolve symbol collisions across the source papers, Opseeq adopts the following canonical notation:

Symbol	**Name**	**Definition**
$s = (\mathbf{req}, \mathbf{ctx}, \mathbf{out})$	Project state	Requirements, accumulated context, partial outputs
$G_\iota = (N, E)$	Intuition graph	DAG of tasks, decisions, outputs, and dependencies
$G_\tau = (V_\tau, E_\tau)$	Trace graph	Directed weighted graph of inference stages
$\mathcal{G} = (V, E, \tau, \phi)$	Artifact graph	Typed temporal multigraph for persistent memory
$M = (H, S, P, \delta, h_0, F)$	Workflow DFA	Deterministic finite automaton for process orchestration
(S, A, T, R, γ)	Workflow MDP	Markov decision process for dynamic optimization
π_θ	Policy model	Local small model as the sole policy-compression layer
$v = f(x; \theta)$	Embedding	Neural map from data to $\mathbb{R}^d$
α_i	Artifact	Durable record $(x_i, y_i, p_i, c_i, t_i, h_i)$
σ	Validity score	Canonical score combining structural validity and invariant coverage
τ	Threshold	Pruning or acceptance threshold (context-dependent)
Q	Query set	Set of user or system queries
P	Prompt set	Set of prompt modules
Σ	Controller state	HPC-GoT typed state tuple

The key reconciliation: the source papers used G for three distinct graph types (trace graph in TraceRank, intuition graph in Context/Process/Query Engine, artifact graph in Cellar). This paper distinguishes them as G_τ, G_ι, and $\mathcal{G}$ respectively.

3.2 Core Axioms

The following axioms constitute the irreducible commitments of the Opseeq framework.

Axiom 1 (Inference Primacy). *The local model π_θ is the sole policy-compression layer within the protocol boundary. All scheduling, retrieval, escalation, and action-selection decisions must enter the system through π_θ or through deterministic transformations of its outputs. External models are optional and explicitly out-of-trust.*

Source: `cellar.tex` *Axiom 1;* `super-intel-desktop.tex` *§Executive Summary.*

Axiom 2 (Artifact Immutability). *Every computationally expensive operation emits a typed, provenance-preserving artifact $\alpha_i = (x_i, y_i, p_i, c_i, t_i, h_i)$. Once written, an artifact is immutable. Corrections are recorded as new artifacts linked through typed provenance edges.*

Source: `cellar.tex` *Axiom 2.*

Axiom 3 (State Triadicity). *Every Opseeq operation is grounded in the triadic state decomposition*

$$\mathrm{Triad}(\mathbf{req}) = (\mathrm{Context}(\mathbf{req}),\ \mathrm{Process}(\mathbf{req}),\ \mathrm{Query}(\mathbf{req})),$$

where the Context Engine manages semantic memory, the Process Engine manages execution orchestration, and the Query Engine manages interpretation and data access.

Source: `query-rqlh.tex` *§framed box;* `query-engine.tex` *§1.*

Axiom 4 (Bounded Hierarchical Compute). *Compute is allocated hierarchically: broad low-cost exploration first, selective deep search second, and strong isolation only where trust boundaries require it. No reasoning controller may exceed its declared state budget.*

Source: `cellar.tex` *Axiom 4;* `HPC-GoT.tex` *Axiom [Global budget].*

Axiom 5 (Harmony-Aware Runtime). *When the base model is a Harmony-trained model served through Ollama, Opseeq records behavior at the API and stream layer while assuming Harmony compliance is enforced upstream by the serving stack. Opseeq does not re-implement Harmony formatting.*

Source: `tracerank.tex` *§Harmony-aware runtime interpretation.*

Axiom 6 (Deterministic External Validation). *Where outputs admit deterministic external validation (rendering, parsing, model checking), structural validity is computed by an external deterministic validator, not by model self-evaluation.*

Source: `HPC-GoT.tex` *Axiom [External validator determinism].*

3.3 Core Postulates

Postulate 1 (Tiered Lakehouse Memory). *Persistent memory is physically partitioned into a hot plane $\mathcal{H}$ (transient, low-latency) and a cold plane $\mathcal{C}$ (immutable, durable), while logically unified by the typed temporal artifact graph $\mathcal{G}$.*

Source: `cellar.tex` *Postulate 1.*

Postulate 2 (Contextual Relevance). *For any query q and document d, relevance is measured by cosine similarity in embedding space:*

$$\mathrm{sim}(q, d) = \cos(v_q, v_d) = \frac{v_q \cdot v_d}{\|v_q\|\,\|v_d\|} \in [0, 1].$$

Retrieved context combines structural and semantic sources:

$$\mathbf{ctx}_q = \mathrm{Cypher}(G_\iota, q) \cup \mathrm{HNSW}(v_q, D).$$

Source: `query-engine.tex` *Postulate 2;* `context-engine.tex` *§3.3.*

Postulate 3 (Calibrated Posterior Output)**.** *The terminal output of any deep reasoning process is a posterior decision summary* $P = \{p_k, \text{paths}_k, \text{policies}_k, \text{intervals}_k\}$ *with uncertainty, ranked explanatory paths, and replayable provenance. Outputs are probabilistic and must not be represented as unconditional truth.*

Source: `cellar.tex` *Postulate 4.*

Postulate 4 (Budgeted Metareasoning)**.** *Each pending computational job* j *is assigned a scalar utility score*

$$J(j) = \mathbb{E}[\Delta U \mid j] - \lambda_c C(j) - \lambda_m M(j) - \lambda_l L(j),$$

and the scheduler selects $j^* = \arg\max_j J(j)$ *subject to hard budget constraints* $\mathcal{B} = (C_{\max}, M_{\max}, L_{\max})$.

Source: `cellar.tex` *Postulate 3.*

3.4 Key Definitions

Definition 1 (Local Singularity)**.** *A model inference run exhibits* local singularity *when* $\text{rank}(J_F(x)) < r_*$, *where* J_F *is the Jacobian of the observable transformation chain and* r_* *is the expected operational rank. This indicates structural collapse detectable through observable rank surrogates.*

Source: `tracerank.tex` *§Mathematical model.*

Definition 2 (Artifact)**.** *An artifact is the canonical durable record of a computationally expensive operation:* $\alpha_i = (x_i, y_i, p_i, c_i, t_i, h_i)$, *where* x_i *is the input signature,* y_i *the output,* p_i *the provenance chain,* c_i *the cost profile,* t_i *the timestamp, and* h_i *the cryptographic hash.*

Source: `cellar.tex` *Definition 2.*

Definition 3 (Temporal Action)**.** *A temporal action is a state transition* $T_t : (s_t, a_t) \mapsto (s_{t+1}, r_t, \omega_t)$ *with canonical fingerprint* $f_t = H(\text{canonical}(s_t, a_t, r_t, \omega_t))$.

Source: `cellar.tex` *Definitions 3 and Postulate 2.*

3.5 Lemmas and Corollaries

Lemma 1 (Artifact Monotonicity)**.** *If artifacts are immutable and all corrections are recorded as appended superseding artifacts, then the epistemic history of the system remains replayable.*

Source: `cellar.tex` *Lemma [Artifact Monotonicity].*

Lemma 2 (Policy-Externalized Capability)**.** *If* θ *is fixed and any of retrieval quality* R, *simulation quality* S, *artifact memory quality* A, *or budgeting quality* B *improves, then effective capability* $\mathcal{I}_{\text{eff}} \approx f(\theta, R, S, A, B)$ *may increase without retraining.*

Source: `cellar.tex` *Lemma [Policy-Externalized Capability].*

Lemma 3 (Query Completeness)**.** *For every query* $q \in Q$, *there exists an execution plan* $\text{Plan}(q) = [o_i]_{i=1}^n$ *such that* $\bigcup_{i=1}^n o_i$ *covers* $\text{Intent}(q)$.

Source: `query-engine.tex` *Lemma 1.*

Corollary 1 (Scalable Retrieval)**.** $\text{Time}(\text{Retrieve}(q)) = O(\log n)$ *where* $n = |\mathbf{ctx}|$, *achieved through HNSW indexing.*

Source: `query-engine.tex` *Corollary 2;* `context-engine.tex` *§3.2.*

4 Full Architectural Overview

Opseeq is organized as a seven-layer architecture. Each layer depends on the layers below it and exposes well-defined interfaces to the layers above.

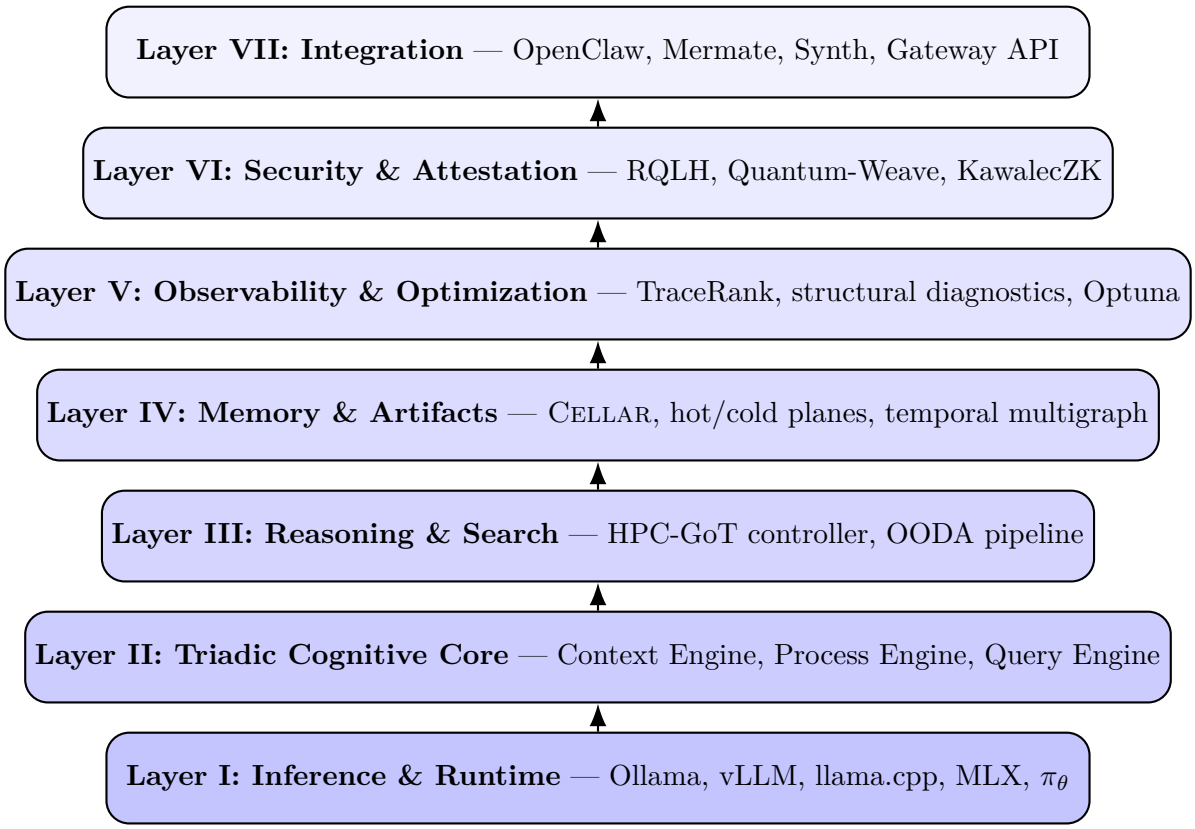

Figure 1: The seven-layer Opseeq architecture. Each layer depends on the layers below it.

5 Layer I: Inference and Runtime

The inference layer provides the raw computational substrate. It hosts one or more local language models and exposes them through standardized APIs.

Primary engines: Ollama (pull-and-run ergonomics, Harmony compliance), vLLM (high-throughput batching), llama.cpp (maximal portability), MLX (Apple Silicon unified memory).

Model substrate: Opseeq is designed around open-weight models in the 7B–21B active-parameter range, with `gpt-oss-20b` (21B parameters, 3.6B active, MoE architecture) as the canonical reference target.[2]

Deployment principle: The inference engine is *not* replaced by Opseeq. It is wrapped. Opseeq treats inference as an observable process over documented surfaces—streamed deltas, structured outputs, runtime timings—rather than as a black box producing final strings.[3]

6 Layer II: The Triadic Cognitive Core

The triadic core is the cognitive center of Opseeq. It decomposes every operation into three complementary engines that jointly manage the full lifecycle from query interpretation through execution.

[2]Source: `tracerank.tex` §3; `HPC-GoT.tex` §1.
[3]Source: `tracerank.tex` §Design thesis.

6.1 Context Engine

The Context Engine maintains the complete project state $s = (\mathbf{req}, \mathbf{ctx}, \mathbf{out})$ using a hybrid graph-vector store.[4]

Components:

- **Context Store.** A directed intuition graph $G_\iota = (N, E)$ for structural relationships plus a vector database D for semantic embeddings.
- **Research Module.** Detects knowledge gaps and issues targeted queries via MCP to enrich the context store.
- **Documentation Generator.** Traverses and formats stored context into structured reports.

Retrieval model:

$$\mathbf{ctx}_q = \mathrm{Cypher}(G_\iota, q) \cup \mathrm{HNSW}(v_q, D),$$

combining structural graph traversal with semantic nearest-neighbor search in $O(\log n)$ time.

State update: After each prompt execution producing output $\hat{y}_p$:

$$\mathbf{ctx}' = \mathbf{ctx} \cup \{\hat{y}_p\}, \quad G'_\iota = G_\iota \cup \{n_{\hat{y}_p}, (n_i \to n_{\hat{y}_p}) \mid n_i \in \mathrm{Prereqs}(\hat{y}_p)\}, \quad D' = D \cup \{v_{\hat{y}_p}\}.$$

6.2 Process Engine

The Process Engine orchestrates workflow execution through a deterministic finite automaton enhanced with MDP-style policy optimization.[5]

DFA model:

$$M = (H, S, P, \delta, h_0, F),$$

where H is the set of workflow stages (Planning, Implementation, Testing, Deployment), P is the prompt set, and δ defines transitions conditioned on validation:

$$h' = \begin{cases} \mathrm{next}(h), & \text{if } \mathrm{Validate}(\hat{y}_p, s) = \mathrm{pass} \wedge \mathrm{StageComplete}(h, s) \\ h, & \text{otherwise.} \end{cases}$$

Prompt selection policy:

$$\mu_h(s) = \arg \max_{p \in \mathrm{Ready}(P_h)} R(s, p),$$

where readiness is determined by dependency completion in G_ι:

$$\mathrm{Ready}(P_h) = \{p \mid \forall n \in \mathrm{Deps}(p),\ n \in \mathrm{Completed}(\mathbf{ctx})\}.$$

MDP formulation for adaptive optimization:

$$\pi^*(s) = \arg\max_\pi \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t))\right], \quad \gamma \in [0, 1).$$

[4]Source: `context-engine.tex` §1–§4.

[5]Source: `process-engine.tex` §1–§8.

6.3 Query Engine

The Query Engine interprets queries from users, agents, or external systems and translates them into operations across the Context and Process Engines.[6]

Pipeline:

$$q \xrightarrow{\text{Parse}} \text{ParsedQuery} \xrightarrow{\text{Plan}} \text{ExecutionPlan} \xrightarrow{\text{Execute}} \{y_i\} \xrightarrow{\text{Format}} r.$$

N2LF Compiler: Converts natural language to logical form:

$$q_{\text{NL}} \to \{\text{intent} : i,\ \text{entities} : E,\ \text{structured_ops} : O\}.$$

Hybrid DBMS: The Query Engine integrates DuckDB (local analytics), Pinecone/FAISS (vector search), and Neo4j (graph traversal) behind a unified data-access abstraction:

$$\text{Retrieve}(q) = \text{Cypher}(G_\iota, q) \cup \text{HNSW}(v_q, D) \cup \text{Analytics}(t, p).$$

6.4 Triadic Integration

The three engines are not independent services; they are tightly coupled through shared state:

$$\text{Triad}(\mathbf{req}) = \big(\text{Context}(\mathbf{req}),\ \text{Process}(\mathbf{req}),\ \text{Query}(\mathbf{req})\big).$$

The Context Engine provides memory. The Query Engine provides interpretation and data access. The Process Engine provides orchestration and execution. Every state transition in the Process Engine reads from the Context Engine and may be initiated by the Query Engine. Every query result from the Query Engine is written back into the Context Engine. This creates a closed loop of interpretation, execution, and learning.

7 Layer III: Reasoning and Search

7.1 The OODA Orchestration Pipeline

Opseeq enforces a mandatory four-phase pipeline for every non-trivial task, following the OODA (Observe–Orient–Decide–Act) pattern:[7]

1. **Observe (Elevation).** Convert raw intent into a complete, unambiguous problem statement.
2. **Orient (Exploration).** Explore the solution space, gather evidence, propose alternatives.
3. **Decide (Consolidation).** Reduce noise, formalize a specification, verify coherence.
4. **Act (Execution).** Produce deliverables and execute authorized actions through guardrailed tools.

The phases are implemented as four agent identities sharing a single base-model substrate. Differentiation is operational—through instruction overlays and routing—not through separate model weights.[8]

[6] Source: `query-engine.tex` §2–§7.
[7] Source: `qwen-4-agents-orch.tex` §5; `super-intel-desktop.tex` §2.2.
[8] Source: `qwen-4-agents-orch.tex` §4.

7.2 HPC-GoT: The Bounded Reasoning Controller

For tasks whose outputs admit deterministic external validation (Mermaid diagrams, PlantUML, TLA$^+$ modules), Opseeq employs HPC-GoT, a bounded typed state-transition controller.[9]

Controller parameters:

- Depth cap: $\ell \in \{0, 1, 2, 3\}$
- Branching: at most 3 children per node
- Global state budget: $N = 40$
- Pruning threshold: $\tau = 0.85$

Canonical validity score:

$$\sigma(A, I, T) = \tfrac{1}{2}\,\mathrm{SV}(A, T) + \tfrac{1}{2}\,\mathrm{IC}(A, I), \tag{1}$$

where $\mathrm{SV} \in \{0, 1\}$ is structural validity (external validator) and IC is invariant coverage.

Operator algebra: Branch, Decompose, Validate, Prune, Select, and an optional single Merge at termination.

Theorem 1 (Termination and Boundedness)**.** *Under the canonical schedule, HPC-GoT terminates after at most three level transitions with* $|\Sigma_{\text{created}}| \leq 40$ *and at most four LLM-mediated stages.*

Source: `HPC-GoT.tex` *Theorems 1–5.*

7.3 Reconciliation: Determinism vs. Stochastic Inference

The Query Engine's Axiom 3 assumes process determinism,[10] while TraceRank explicitly measures run-to-run instability D_{run}.[11] Opseeq resolves this as follows:

Remark 1 (Conditional Determinism)*.* Process determinism holds for *validated* execution paths: once an output passes validation ($\mathrm{Validate}(\hat{y}_p, s) = \mathrm{pass}$), its state-update effect is deterministic. The stochastic variability of the model's generation process is an input to TraceRank's observability layer, not a contradiction of the Process Engine's execution model.

8 Layer IV: Memory and Artifacts

8.1 The CELLAR Runtime

Cellar (Cognitive Emergent Local Lakehouse Artifact Runtime) is the memory substrate of Opseeq.[12] Its central thesis is that a small model should not be treated as the whole of intelligence, but as a bounded policy layer governing retrieval, scheduling, selective simulation, and durable memory formation.

Five formal layers:

1. **Policy Layer:** $\pi_\theta(o_t, m_t, b_t) \mapsto (u_t, q_t, \hat{V}_t, \hat{C}_t)$
2. **Transition Layer:** $T_t : (s_t, a_t) \mapsto (s_{t+1}, r_t, \omega_t)$
3. **Artifact Layer:** $\alpha_i = (x_i, y_i, p_i, c_i, t_i, h_i)$

[9] Source: `HPC-GoT.tex` §1–§9.
[10] `query-engine.tex` §6.1.
[11] `tracerank.tex` §Mathematical model.
[12] Source: `cellar.tex` §1–§12.

4. **Graph Layer:** $\mathcal{G} = (V, E, \tau, \phi)$ — typed temporal multigraph
5. **Posterior Layer:** $P = \{p_k, \text{paths}_k, \text{policies}_k, \text{intervals}_k\}$

Each layer depends on the previous: inference proposes actions, actions generate transitions, transitions emit artifacts, artifacts populate the graph, and graph-grounded trajectories collapse into posterior summaries.

8.2 Hot and Cold Storage Planes

Hot plane $\mathcal{H}$: Transient execution-near batches, queue-local records, vectorized summaries awaiting compaction. Optimized for low-latency access and bounded lifetime.

Cold plane $\mathcal{C}$: Immutable artifacts, manifests, indices, checkpoints, provenance records. Optimized for durability, replayability, and structured querying.

Practical substrate: DuckDB for local analytical queries over artifacts; LanceDB or FAISS for vector indices; Apache Arrow for columnar serialization.

8.3 Deduplication Pipeline

Artifact insertion proceeds through: (1) exact hash lookup, (2) semantic candidate retrieval, (3) type-aware merge-or-append decision, (4) hot-plane insertion, (5) scheduled flush to cold storage, (6) graph-edge materialization and provenance update.[13]

9 Layer V: Observability and Optimization

9.1 TraceRank

TraceRank is a sidecar observability and optimization system that transforms local inference from a black-box loop into a measurable, benchmarkable, and tunable runtime.[14]

Observable transformation chain:

$$x \xrightarrow{T_1} h_1 \xrightarrow{T_2} \cdots \xrightarrow{T_L} h_L = z,$$

where h_k are observable stage representations and z is the final trace vector.

Concentration:

$$C(z) = 1 - \frac{H(z)}{\log d}, \quad H(z) = -\sum_{i=1}^{d} p_i \log p_i.$$

Graph collapse:

$$\gamma_{A_\tau} = 1 - \frac{\text{rank}(A_\tau)}{|V_\tau|},$$

where A_τ is the adjacency matrix of the trace graph G_τ.

Composite structural score:

$$S = \omega_1 C(z) + \omega_2 \gamma_{A_\tau} + \omega_3 \kappa_{\text{obs}} + \omega_4 D_{\text{run}}, \tag{2}$$

where high S indicates elevated collapse risk, narrow routing, or unstable behavior.

Optimization objective:

$$J(\theta) = \lambda_1 Q(\theta) - \lambda_2 L(\theta) - \lambda_3 M(\theta) - \lambda_4 V(\theta), \tag{3}$$

balancing quality Q, latency L, memory pressure M, and instability V.

[13] Source: `cellar.tex` §Implementation Notes.
[14] Source: `tracerank.tex` §1–§14.

9.2 Adapter Promotion Pipeline

TraceRank defines four adapter states—Candidate, Validated, Shadow-tested, Promoted—with promotion requiring improvement over baseline, no schema-fidelity regression, no material latency/memory regression, and a rollback-safe shadow test.[15]

10 Layer VI: Security and Attestation

10.1 RQLH Consensus

The Relativistic Quantum Lattice Hash provides collision-resistant attestation for the assembly process:[16]

$$\Pr[H(x) = H(y)] \leq \left(\frac{d}{q_*}\right)^k,$$

where d is polynomial degree, q_j are prime moduli, and $q_* = \min_j q_j$. This bound follows from the Schwartz–Zippel lemma applied over a CRT lattice structure.

10.2 Quantum-Weave: Post-Quantum Data Security

For data-at-rest and data-in-transit security, Opseeq adopts the Quantum-Weave framework:[17]

- **Key exchange:** CRYSTALS-Kyber (M-LWE-based, quantum-resistant).
- **Symmetric encryption:** AES-256-GCM (128-bit effective quantum security via Grover's bound).
- **Integrity:** SHA-256 Merkle trees providing $O(\log n)$ verification.
- **Authentication:** CRYSTALS-Dilithium digital signatures on Merkle roots.

10.3 Trust Boundary Model

Opseeq distinguishes:[18]

- **In-trust:** Local inference, local orchestration, formal checking, local testing, local artifact storage.
- **Out-of-trust (optional):** Any external model API, cloud oracle, or remote service. Used only for critique/triangulation, never for authoritative control.

11 Layer VII: Integration — OpenClaw, Mermate, and Synth

Opseeq functions as the super-computational and orchestration framework underlying three primary application surfaces.

11.1 OpenClaw

OpenClaw is the execution sandbox and agent runtime. Opseeq provides OpenClaw with:

- Inference routing through the gateway API (`/v1/chat/completions`, `/api/chat`).
- Network policy management and sandboxed execution boundaries.
- MCP tool exposure for agentic control.

[15] Source: `tracerank.tex` §Adapter and LoRA workflow.
[16] Source: `query-rqlh.tex` §Cryptographic Foundations.
[17] Source: `lattice-opseeq.tex` §4–§7.
[18] Source: `super-intel-desktop.tex` §1.

11.2 Mermate

Mermate is the architecture and specification copilot. It is the primary consumer of HPC-GoT's validator-backed reasoning for Mermaid, PlantUML/C4, and TLA$^+$ artifact generation. Opseeq provides Mermate with:

- Triadic engine access for context-aware specification generation.
- Architecture pipeline orchestration (`/api/architect/pipeline`).
- Formal specification rendering (TLA$^+$/TypeScript).

11.3 Synth (Synthesis-Trade)

Synth is a prediction market trading desk. Opseeq provides Synth with:

- Inference routing and model access through the OpenAI-compatible gateway.
- Health monitoring and availability status via background polling.
- Integration hooks through MCP tools for AI-powered market analysis.

11.4 Gateway Implementation

The Opseeq gateway (currently `opseeq:latest`, v2.1.0) exposes:[19]

- OpenAI-compatible API: `/v1/models`, `/v1/chat/completions`
- Console-compatible API: `/api/status`, `/api/chat`, `/api/connectivity`
- MCP server: 15 tools over SSE at `/mcp`
- Multi-provider inference routing: NVIDIA NIM, OpenAI, Anthropic, Ollama

12 Assembly and Implementation Logic

12.1 How the System Is Constructed

Opseeq is assembled as a set of cooperating processes coordinated by Docker Compose on a single workstation:

1. **Inference engine:** Ollama or vLLM serves the base model locally.
2. **Opseeq gateway:** Node.js/Express service providing the triadic API surface, inference routing, MCP, and application integration.
3. **Application surfaces:** Mermate (Node.js, port 3333) and Synth (Node.js/Python, port 8420) connect to Opseeq as downstream consumers.
4. **Storage:** Local DuckDB for analytical queries, LanceDB/FAISS for vector indices, filesystem for artifact cold storage.

[19] Status: **Implemented**. See `service/src/index.ts`.

12.2 Runtime Wiring

At startup, the gateway:

1. Parses provider configuration from environment variables.
2. Initializes HTTP keep-alive agents for upstream connections.
3. Starts background health polling for Mermate and Synth.
4. Initializes timed caches for status, model lists, and service state.
5. Registers graceful shutdown handlers (SIGTERM/SIGINT) with connection draining.
6. Exposes the MCP server with 15 registered tools.

12.3 The Assembly Loop

For full-stack development tasks, Opseeq executes the assembly loop formalized in the RQLH paper:[20]

$$\begin{aligned}\text{Assembly}(s) = \text{loop}\Big(\; & p = \text{select}(R, \mu(\cos));\\ & \hat{y}_p = \text{llm}(p, X = \mathbf{req}_p + \mathbf{ctx}_q + \text{mcp});\\ & \text{if } v(\hat{y}_p) < \tau_v \text{ then refine}(\hat{y}_p) \text{ else update}(s', G_\iota \cup \{\hat{y}_p\});\\ & \text{a2a}(\hat{y}_p);\; R.\text{upsert}\;\Big),\end{aligned}$$

where $v(\hat{y}_p) = \text{test} + \cos + \text{review}$ and τ_v is the validation threshold.

13 Temporal State Sequencing and System Behavior Over Time

Opseeq's behavior over time is governed by three interacting temporal processes:

1. **Workflow state transitions:** The Process Engine's DFA advances through stages $(h, s) \xrightarrow{p} (h', s')$ as prompts execute and validate.
2. **Memory accumulation:** The artifact graph $\mathcal{G}$ grows monotonically. Hot-plane entries are periodically flushed to cold storage. The intuition graph G_ι expands with each validated output.
3. **Observability evolution:** TraceRank's structural score S evolves across runs, detecting behavioral drift. The adapter promotion pipeline moves candidates through states over time: Candidate → Validated → Shadow-tested → Promoted.

Convergence termination: Following Cellar's posterior convergence postulate, deep reasoning may terminate when either the hard budget is exhausted or successive posterior estimates stabilize:

$$\text{KL}(P_t \| P_{t+1}) \leq \varepsilon.$$

[20] Source: `query-rqlh.tex` §Assembly.

14 Human-in-the-Loop and Tool-Mediated Control

14.1 The Human Invariant Layer

Following the APCL methodology,[21] the human is the sole author of the immutable foundation of correctness:

- **Axioms:** Non-negotiable truths and constraints.
- **Postulates:** Assumed properties and operating conditions.
- **Corollaries:** Derived constraints that must hold.
- **Lemmas:** Intermediate claims required for proofs.

The system traces every generated artifact back to these human-authored invariants.

14.2 Guardrailed Tool Execution

Tool access follows the sandbox model:[22]

- User specifies allowed root directories at launch.
- All file operations resolve absolute paths and confirm containment.
- Subprocess commands use `shell=False` with explicit timeouts.
- Permission escalation requires explicit user approval.

14.3 Tiered Validation Policy

The source papers used multiple thresholds ($\tau = 0.8$ in RQLH, $\tau = 0.85$ in HPC-GoT, coverage ≥ 0.9 in testing stage). Opseeq unifies these into a tiered policy:

Tier	Threshold	Application
Exploration	$\tau_e = 0.7$	Broad-phase search, initial retrieval filtering
Production	$\tau_p = 0.85$	HPC-GoT pruning, assembly validation
Deployment	$\tau_d = 0.9$	Testing stage, deployment gate

15 Performance and Local Runtime Model

Opseeq is designed for single-workstation deployment. Its performance model makes the following commitments:

Inference: The base model runs on consumer hardware (Apple Silicon with 32–64 GB unified memory, or x86 with GPU). Weight memory is approximately Params $\times$ bits/8 bytes; practical context windows are constrained by KV-cache memory.[23]

Retrieval: $O(\log n)$ via HNSW for vector search; $O(|N| + |E|)$ for topological sort on the intuition graph.

HPC-GoT: $O(1)$ controller-state complexity (budget is a fixed constant $N = 40$); at most 4 LLM-mediated stages under the canonical batching contract.

Caching: LRU caches with timed expiry for frequently accessed state (model lists, service health, connectivity probes).

[21] Source: `super-intel-desktop.tex` §2.1.
[22] Source: `qwen-4-agents-orch.tex` §8.
[23] Source: `qwen-4-agents-orch.tex` §10.

Network: HTTP keep-alive agents for upstream provider connections. Background polling with exponential backoff for service health.

Optimization: Offline multi-objective search via Optuna over the configuration space (context length, concurrency, temperature, stop conditions).

16 Reliability and Production Constraints

Graceful shutdown: SIGTERM/SIGINT handlers with connection draining and configurable grace period.

Fault tolerance: Checkpoint-restart for workflow state; retry with exponential backoff for transient upstream failures; rollback to previous adapter/configuration baseline.

Serverless idle: Optional idle-timeout shutdown for resource deactivation when no active connections exist.

Health monitoring: `/health` (basic), `/health/ready` (deep readiness), and background polling of dependent services.

Limitations:

- TraceRank does not expose hidden activations through Ollama's standard API. It uses observable rank surrogates.
- HPC-GoT guarantees controller-level properties, not semantic correctness.
- Cellar's posterior quality depends on retrieval, simulation, and scoring quality.
- Opseeq does not claim to replace policy enforcement, moderation, or content safety systems.

17 Final Synthesis

Opseeq is a seven-layer intelligence architecture that transforms a developer's workstation into a sovereign, formally grounded reasoning system. Its design proceeds from a single organizing principle: *the language model is the policy layer, not the system.*

Around that policy layer, Opseeq constructs:

1. A **triadic cognitive core** (Context, Process, Query) that manages the complete lifecycle from interpretation through execution, grounded in hybrid graph-vector memory and formal state machines.
2. A **bounded reasoning controller** (HPC-GoT) that structures search-and-validate over architecture artifacts with provable termination, bounded state, and external validator scoring.
3. An **artifact-centric memory runtime** (Cellar) that makes expensive computation durable, typed, and replayable through a temporal multigraph over hot and cold storage planes.
4. A **sidecar observability layer** (TraceRank) that detects structural degradation before quality visibly declines and supports gated adapter promotion.
5. A **post-quantum attestation substrate** (RQLH and Quantum-Weave) that ensures integrity under both classical and quantum threat models.
6. An **OODA orchestration pipeline** that enforces disciplined transformation from raw intent through verified execution.

7. An **integration gateway** that connects OpenClaw, Mermate, and Synth through a unified, multi-provider API surface.

The architecture is local-first but not local-only. It is formally grounded but not divorced from implementation. It is artifact-centric but not storage-obsessed. It makes the path from question to answer inspectable, replayable, and revisable—not through one model call, but through structured computational inquiry.

Opseeq does not claim omniscience. It claims something more disciplined: that a small model, embedded in a well-designed runtime, can become substantially more capable than the same model prompted in isolation. The improvement is not merely in accuracy. It is in auditability, reproducibility, safety, and the scientific integrity of the reasoning process itself.

A Source Cross-Reference Matrix

The following table maps each section of this master document to the source papers that contributed to it.

Section	Primary Sources
§1 Introduction	`super-intel-desktop.tex`, `tracerank.tex`, `cellar.tex`
§2 Problem Framing	`super-intel-desktop.tex` §1, `qwen-4-agents-orch.tex` §2
§3 Theoretical Foundations	`cellar.tex` §3–4, `query-engine.tex` §6, `tracerank.tex` §5, `HPC-GoT.tex` §4, `query-rqlh.tex`
§4 Architecture Overview	All source papers (synthesis)
§5 Layer I: Inference	`tracerank.tex` §3–6, `super-intel-desktop.tex` §3, `qwen-4-agents-orch.tex` §4
§6 Layer II: Triadic Core	`context-engine.tex` §1–5, `process-engine.tex` §1–8, `query-engine.tex` §1–9, `query-rqlh.tex`
§7 Layer III: Reasoning	`HPC-GoT.tex` §1–9, `qwen-4-agents-orch.tex` §5–6, `super-intel-desktop.tex` §2
§8 Layer IV: Memory	`cellar.tex` §1–12
§9 Layer V: Observability	`tracerank.tex` §5–11
§10 Layer VI: Security	`query-rqlh.tex` §4, `lattice-opseeq.tex` §4–7, `super-intel-desktop.tex` §2.4
§11 Layer VII: Integration	`query-rqlh.tex` §2, implemented gateway (`service/src/`)
§12 Assembly Logic	`query-rqlh.tex` §Assembly, implemented gateway
§13 Temporal Sequencing	`process-engine.tex` §4, `cellar.tex` §6, `tracerank.tex` §10
§14 HITL Control	`super-intel-desktop.tex` §2.1, `qwen-4-agents-orch.tex` §8
§15 Performance Model	`qwen-4-agents-orch.tex` §10, `HPC-GoT.tex` §App. Complexity, `tracerank.tex` §9
§16 Reliability	`process-engine.tex` §7, `cellar.tex` §11, implemented gateway

B Unresolved Source

The source paper `agent-builder-framework.tex` was listed in the plan but does not exist in the repository at `docs/wp/`. Its intended content—an agent-builder framework specification—is partially covered by the OODA orchestration pipeline (§7.1), the HPC-GoT controller (§7.2), and the Cellar runtime (§8.1). If this paper is later written, its content should be integrated into Layers II–III of this master document.

C Notation Index

Symbol	Meaning
$s = (\mathbf{req}, \mathbf{ctx}, \mathbf{out})$	Project state: requirements, context, outputs
$G_\iota = (N, E)$	Intuition graph (task/dependency DAG)
$G_\tau = (V_\tau, E_\tau)$	Trace graph (inference-stage graph)
$\mathcal{G} = (V, E, \tau, \phi)$	Typed temporal artifact multigraph
$\mathcal{H}$, $\mathcal{C}$	Hot and cold storage planes
π_θ	Local policy model with parameters θ
$v = f(x; \theta)$	Neural embedding function
α_i	Artifact: $(x_i, y_i, p_i, c_i, t_i, h_i)$
σ	Validity score: $\frac{1}{2}\,\mathrm{SV} + \frac{1}{2}\,\mathrm{IC}$
S	TraceRank structural score
$J(\theta)$	Optimization objective
Σ	HPC-GoT controller state
$N = 40$	HPC-GoT global state budget
$\mathcal{I}_{\text{eff}}$	Effective system-level intelligence

References

[1] D. Kawalec, "TraceRank v3: A Harmony-Aware Observability and Optimization Layer for gpt-oss-20b on Ollama," internal whitepaper, March 2026. [`tracerank.tex`]

[2] D. Kawalec, "Sovereign Desktop Superintelligence: Local-First Formal-Agentic Engineering Pipeline," chaotic beauty LLC, March 2026. [`super-intel-desktop.tex`]

[3] "Qwen3.5-9B Four-Agent OODA Orchestration Framework (Version 1.0)," March 2026. [`qwen-4-agents-orch.tex`]

[4] D. C. Kawalec, "Opseeq: A Sovereign Local-First AI Framework for Prompt-Driven, Verifiable Full-Stack Development with Relativistic Quantum Lattice Hash (RQLH) Consensus," September 2024. [`query-rqlh.tex`]

[5] D. Kawalec, "Opseeq Query Engine and Database Management System Architecture," May 2025. [`query-engine.tex`]

[6] D. Kawalec, "The Process API Engine for Opseeq: A Formal Architecture for Orchestrating AI-Driven Development," May 2025. [`process-engine.tex`]

[7] D. Kawalec, "Quantum-Weave: A Lattice-Based Cryptographic Framework for Scalable and Secure Data Transmission Between DuckDB and Snowflake," Greybeam.ai, May 2025. [`lattice-opseeq.tex`]

[8] D. Kawalec, "HPC-GoT: A Bounded Hierarchical Reasoning Controller for Local 21B-Scale Mixture-of-Experts Models on Architectural System-Specification Tasks," March 2026. [`HPC-GoT.tex`]

[9] "The Context Engine for Opseeq: A Comprehensive Design for Large-Scale AI Solution Development," May 2025. [`context-engine.tex`]

[10] D. Kawalec, "CELLAR: Cognitive Emergent Local Lakehouse Artifact Runtime," 2026. [`cellar.tex`]

[11] R. Bellman, “A Markovian Decision Process,” *J. Math. Mech.*, vol. 6, no. 5, 1957.

[12] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using HNSW,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.

[13] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, 2022.

[14] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” arXiv:2305.10601, 2023.

[15] M. Besta et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” arXiv:2308.09687, 2023.